A
Project Report
On
DataForge – End-to-End AutoML Platform With Intelligent Data Preprocessing

Prepared by

Harshit Pambhar (24DCS060)
Krish Ramanandi (24DCS107)
Sarthak Talaviya (24DCS131) (TL)
Jemit Vaghasiya (24DCS145)

Under the guidance of

Dr. Parth Goel

Submitted to

Charotar University of Science & Technology

Degree of Bachelor of Technology

in Computer Science and Engineering

CSUP201: SOFTWARE GROUP PROJECT

of 4th Semester of B.Tech

Submitted at

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Faculty of Technology & Engineering, CHARUSAT

Devang Patel Institute of Advance Technology and Research (DEPSTAR)

At: Changa, Dist: Anand – 388421

April 2026

CHARUSAT
CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

CERTIFICATE

This is to certify that the report entitled "DataForge – End-to-End AutoML Platfform with Intelligent Data Preprocessing" is a bonafied work carried out by

- Harshit Pambhar (24DCS060)
- Krish Ramanandi (24DCS107)
- Sarthak Talaviya (24DCS131) (TL)
- Jemit Vaghasiya (24DCS145)

under the guidance and supervision of **Dr. Parth Goel** for the subject **SOFTWARE GROUP PROJECT CSUP201** of 4th Semester of Bachelor of Technology in **Computer Science and Engineering** at Devang Patel Institute of Advance Technology and Research (DEPSTAR), Faculty of Technology & Engineering (FTE) – CHARUSAT, Gujarat

To the best of my knowledge and belief, this work embodies the work of candidate himself, has duly been completed, and fulfills the requirement of the ordinance relating to the B.Tech. Degree of the University and is up to the standard in respect of content, presentation and language for being referred to the examiner.

Under the supervision of

_____

Dr. Parth Goel
Dept. of Computer Science and Engineering,
DEPSTAR, CHARUSAT-Changa.

_____

Dr. Amit Nayak
Head of Department,
Dept. of Computer Science and Engineering,
DEPSTAR, CHARUSAT-Changa.

DECLARATION BY THE CANDIDATES

We hereby declare that the project report entitled "DataForge - End-to-End AutoML Platform with Intelligent Data Preprocessing " submitted by us to Devang Patel Institute of Advance Technology and Research (DEPSTAR), Changa in partial fulfilment of the requirements for the award of the degree of **B.Tech Computer Science and Engineering**, from the Computer Science and Engineering, DEPSTAR, FTE is a record of bonafide CSUP201 SOFTWARE GROUP PROJECT carried out by us under the guidance of Dr. Parth Goel .We further declare that the work carried out and documented in This project report has not been submitted anywhere else either in part or in full and it is the original work, for the award of any other degree or diploma in this institute or any other institute or university.

Signature of Student 1 (Harshit Pambhar) (24DCS060)	Signature of Student 2 (Krish Ramanandi) (24DCS107)
Signature of Student 3 (Sarthak Talaviya) (24DCS131)	Signature of Student 4 (Jemit Vaghasiya) (24DCS145)

This is to certify that the above statement made by the candidate is correct to the bast of my knowledge

Abstract

DataForge is a semi-automated machine learning platform designed to simplify data preprocessing and model training. The system provides a user-friendly interface that allows users to perform preprocessing operations through interactive UI controls, including handling missing values, encoding, and feature selection.

The primary objective of the project is to reduce the complexity of machine learning workflows while maintaining user control over preprocessing steps. After completing preprocessing, users can initiate model training with a single click, where the system automatically trains and evaluates multiple machine learning models.

The platform is developed using React.js for the frontend, Node.js for backend services, FastAPI for machine learning operations, MySQL for data storage, and Redis with BullMQ for asynchronous job processing.

Industry Impact:

DataForge can significantly reduce manual effort in machine learning workflows by approximately 50–70%, while decreasing model training time by 30–40% through automated processing. It enables faster experimentation, improves productivity, and makes machine learning more accessible to non-expert users.

While the current system follows a guided, user-driven approach, future enhancements aim to transform DataForge into a fully automated AutoML platform.

Table of Contents

1. Introduction 1
2. Literature Review 2
3. System Analysis 3
4. Technology Stack 4
5. System Design 5
6. Testing 6
7. Results 7
8. Challenges Faced 8
9. Conclusion and Future Scope 9
10. References 10
11. Appendices 11

CHAPTER 1
Introduction

• Background of the Project

The DataForge project comes from the increasing use of data and machine learning in today's industries. As organizations generate large amounts of data, the need to analyze and gain useful insights has become very important. While machine learning helps in this process, it is not easy to use because it requires knowledge of data preprocessing, feature selection, and model tuning. Due to this complexity, many users such as students and developers find it difficult to apply machine learning in real-life situations.

• Problem Definition

The main problem is that traditional machine learning workflows are complex and time-consuming. They involve multiple steps that must be done manually and require technical knowledge. Tasks like handling missing data, encoding, and normalization can easily lead to errors. Also, these systems are not very efficient when working with large datasets or multiple tasks at the same time. This makes it necessary to develop a system that simplifies the entire process.

• Motivation for the Project

The motivation behind DataForge is to make machine learning easier and more accessible. The goal is to reduce the need for expert knowledge by automating repetitive tasks and providing a simple user interface. By using scalable technologies, the system can handle large data and multiple processes efficiently, allowing users to focus more on results rather than technical details.•

• Objectives and Scope

The objective of DataForge is to build a semi-automated platform where users can easily upload datasets, perform preprocessing through a user interface, and train models with a single click. It also focuses on scalability using queue-based processing and aims to support future AutoML features.

The scope of the project includes developing a web-based system that allows dataset upload, customizable preprocessing, automated model training, result visualization, and exporting of datasets and models. This makes it useful for both learning purposes and real-world applications.

CHAPTER 2
Literature Review

- **Existing Solutions:**
    - Google AutoML
    - H2O.ai
    - Orange
    - Auto-sklearn

- **Comparative Analysis**

Platform	Ease Of Use	Automation Level	Cost	Flexibility	Deployment
Google AutoML	Very High	Fully Automated	Paid (with limited free tier)	Medium	Cloud-based
H2O.ai	Medium	High	Free (core) + Paid (enterprise)	HIgh	Local + Cloud
Orange	High	Low(Manual Workflow)	Free	Medium	Desktop Based
Auto Sklearn	Low	High	Free (Open Source)	High	Local Python Based
DataForge	High	Semi Automated (Ui + Auto Training)	Free Credits-> paid Plan	High	Web-Based

- **Project Contribution:-**

  DataForge differs by:

    - Providing full-stack integration (frontend + backend + ML)
    - Using queue-based job processing (BullMQ)
    - Offering customizable preprocessing pipeline
    - Being cost-effective and open

CHAPTER – 3
System Analysis

- **Functional Requirements**

  - User Registration
  - User Login and Authentication
  - Dataset Upload
  - Dataset Storage and Management
  - Dataset Selection for Processing
  - UI-Based Data Preprocessing Options
  - Data Cleaning (handling missing values)
  - Data Transformation (encoding, scaling)
  - Model Training Initiation (one-click)
  - Automated Model Training Execution
  - Job Queue Management (asynchronous processing)
  - Model Evaluation and Result Display

- **Non – Functional Requirements**

  - Scalability using Redis and BullMQ
  - High Performance for large dataset processing
  - Secure Authentication using JWT
  - Data Security and Privacy
  - Responsive User Interface
  - System Reliability and Availability
  - Fault Tolerance with retry mechanisms

CHAPTER 4
Technology Stack

1. Frontend (UI/UX)

- **React 18** (UI Library)
- **Vite 5** (Build Tool)
- **Tailwind CSS 3** (Styling)
- **React Router DOM 6** (Navigation)
- **TanStack React Query 5** (Data Fetching)
- **Framer Motion / GSAP** (Animations)

2. Backend (API & Logic)

- **Node.js 20 LTS** (Runtime)
- **Express 4** (Web Framework)
- **Multer** (File Upload)
- **JWT** (Authentication)
- **bcryptjs** (Encryption)
- **Axios** (HTTP Client)
- **Socket.io** (Real-time Communication)
- **Cors / Body-parser** (Utilities)

3. ML Service (AI & Analytics)

- **Python** (Programming Language)
- **FastAPI** (Web Framework)
- **Uvicorn** (ASGI Server)
- **Pandas** (Data Manipulation)
- **NumPy** (Numerical Computing)
- **Scikit-learn** (Machine Learning)
- **Pydantic** (Data Validation)
- **Joblib** (Model Saving/Loading)

4. Datastores & Messaging

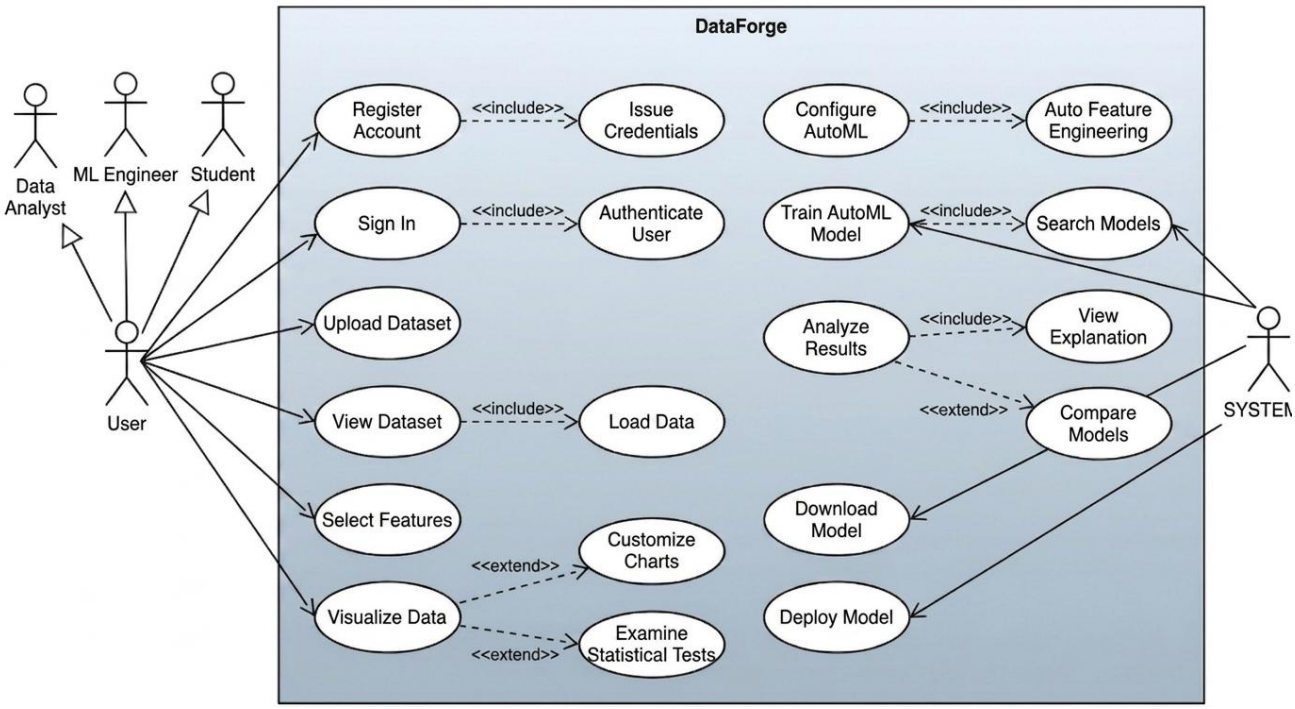
- **MySQL** (Primary Relational Database)
- **Redis** (In-memory Caching & Message Broker)
- **BullMQ** (Task Queue Management)
- **ioredis** (Redis Client)

5. Infrastructure

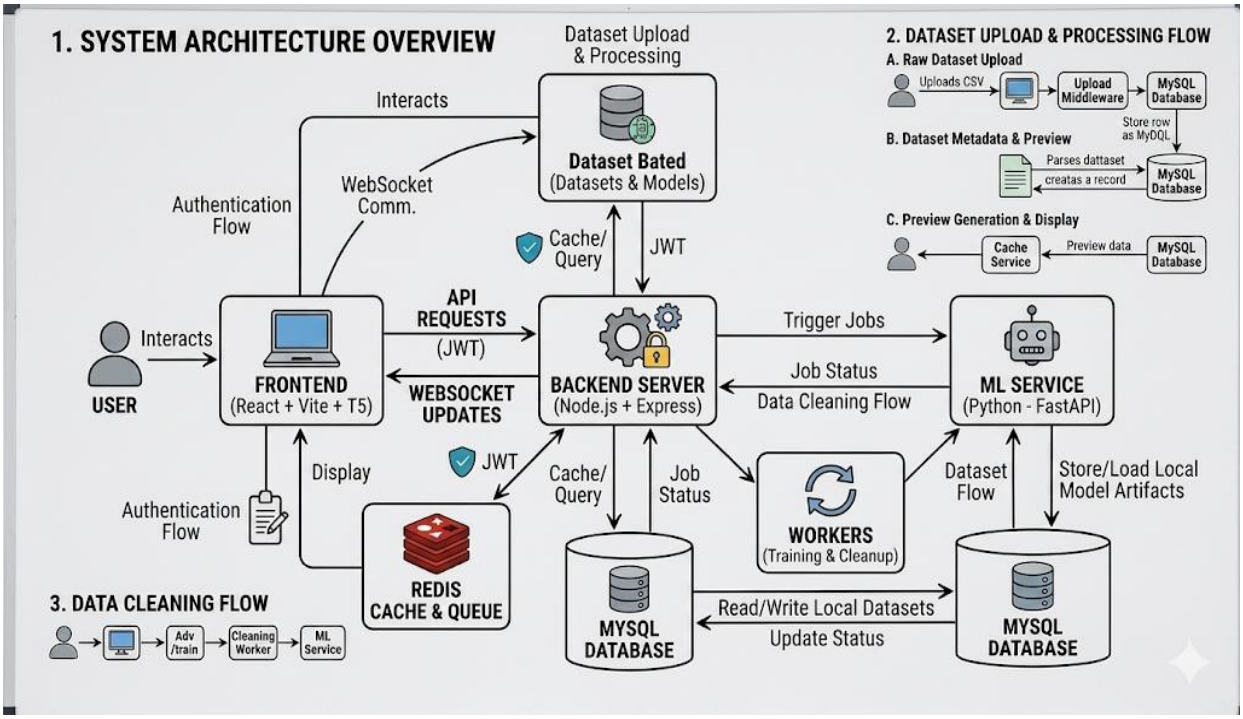
- **Docker** (Containerization)
- **Docker-Compose** (Service Orchestration)

CHAPTER 5
System Design

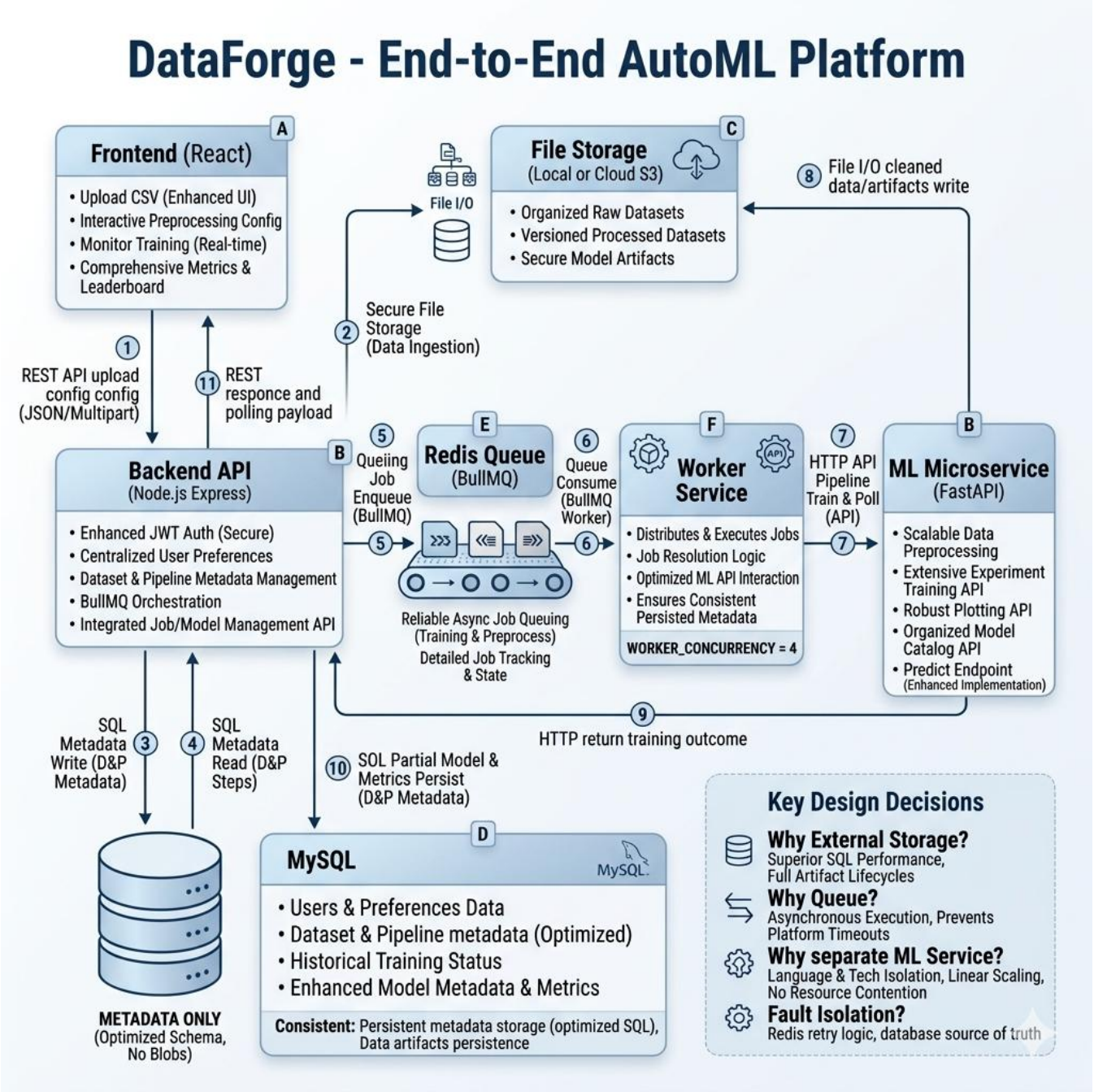
- **Use Case Diagram:**



- **Data Flow Diagram:**



- **Architecture Diagram**



CHAPTER 6
Testing

- **Type Of Testing**

  - **Unit Testing:** Unit testing was performed on individual modules such as:

    - API endpoints
    - Data preprocessing functions
    - Authentication logic
    - Each unit was tested independently to ensure correct functionality.

  - **Integration Testing:** Integration testing verified the interaction between different components:

    - Frontend ↔ Backend
    - Backend ↔ ML Service
    - Backend ↔ Queue System (Redis/BullMQ)
    - This ensured smooth data flow across the system.

  - **System Testing:** System testing evaluated the complete workflow:

    - Dataset upload
    - Data cleaning
    - Model training
    - Result generation
    - The entire system was tested as a whole to validate real-world use cases.

  - **Black Box Testing :** Black box testing was performed to validate system functionality without considering internal implementation:

- **Tools Used**

  - Postman – For testing API endpoints
  - Swagger UI – For API visualization and testing
  - Manual UI Testing – For validating frontend functionality

Test Case	Expected Result	Actual Result	Status
Login	Success	Success	Pass
Invaild Login	Error message	Error Displayed	Pass
Upload Dataset	Not Stored	Not Stored	Pass
Invalid File Upload	Rejected	Rejected	Pass
Data Cleaning	Processed Correctly	Processed Correctly	Pass
Train Model	Job Start	Job Start	Pass
Results	Displayed	Displayed	Pass

CHAPTER 7
Result

1. Home:

The home page provides an overview of the DataForge platform, highlighting its core features such as data cleaning, model training, and analytics. It serves as the entry point where users can explore the platform and initiate the workflow.

2. Signup :

The signup page allows new users to create an account by providing necessary credentials. It ensures secure user registration and enables access to personalized features of the platform.

3. Dataset, Loaded_Dataset:

This page allows users to upload datasets in CSV format and manage existing datasets. Users can also resume previously processed datasets, enabling continuity in workflow.

4. Cleaning:

The data cleaning page provides UI-based preprocessing features such as handling missing values, removing duplicates, encoding categorical data, and feature scaling. It allows step-by-step data transformation with options like undo and preview.

5. Visualization, Charts:

This module allows users to visualize dataset features using graphs such as histogram, box plot, line chart, scatter plot, and heatmap. It helps in understanding data distribution and relationships before model training.

6. Training:

The training page enables users to select target variables, choose problem type (classification/regression), and select models. Model training is initiated with a single click, and jobs are processed asynchronously.

7. Leaderboard:

The leaderboard page displays model performance metrics such as accuracy, RMSE, and R^2 score. It allows users to compare multiple models and identify the best-performing one.

CHAPTER 8
Challenges

1. Preprocessing Without Dataset Storage:

Handling preprocessing without permanently storing datasets was challenging.

Solution :

The system processes data in temporary variable form during execution, while only storing pipeline configurations and results in the database.

2. Pipeline Configuration:

Designing a flexible pipeline where users can configure multiple preprocessing steps dynamically was complex.

Solution :

A modular pipeline structure was implemented, where each step is defined and executed sequentially based on user input.

3. Undo Functionality and Step-wise Data Display:

Providing undo functionality and displaying intermediate results after each preprocessing step required maintaining state.

Solution :

Each preprocessing step is stored as a state, enabling rollback (undo) and allowing users to view updated data after every step.

4. Multiple Pipelines for a Single User:

Allowing users to manage multiple pipelines simultaneously created challenges in tracking and execution

Solution :

Unique pipeline IDs and job tracking mechanisms were implemented to handle multiple pipelines independently.

5. Managing Asynchronous Jobs:

Handling long-running ML tasks without blocking the system was difficult.

Solution :

BullMQ with Redis was used to process jobs asynchronously through background workers.

6. Synchronization Between Services:

Ensuring smooth communication between frontend, backend, and ML service was challenging.

Solution :

REST APIs and Socket.io were used for reliable communication and real-time updates.

7. Database Consistency Issues:

Maintaining consistency across jobs, pipelines, and results was complex.

Solution :

Proper schema design and controlled updates were implemented to ensure data integrity and consistency.

CHAPTER 9
Conclusion and Future Scope

Conclusion :

DataForge provides a semi-automated machine learning platform where users can perform preprocessing through a UI and train models with a single click. The system simplifies the ML workflow while maintaining user control over data processing.

It successfully integrates frontend, backend, and ML services with asynchronous job handling, ensuring efficient and scalable execution.

Future Scope :

- **Full Automation (AutoML)**
  Automate preprocessing and model selection to reduce manual effort by 50–70%.

- **HyperParameter Tuning**
  Improve model accuracy by approximately 10–20% using automated tuning.

- **Parallel Processing**
  Reduce training time by 30–40% through parallel model execution.

- **Large Dataset Support**
  Extend system capability to handle datasets larger than 500MB–1GB.

- **Cloud Deployment**
  Deploy on cloud platforms for better scalability and availability.

CHAPTER 10
REFERENCES

- **Google** AutoML Documentation
- **H2O.ai** Documentation
- **FastAPI** Official Docs
- **React.js** Documentation
- **Node.js** Documentation
- **BullMQ** Documentation
- **Research papers** on AutoML

CHAPTER 11
APPENDICES

➢ Appendix A: DataForge API Documentation *(Source:* API DOCUMENTATION)

A.1 Base PORTs

- Backend: 5000
- ML Service: 8000
- Frontend Dev Server: 5173
- MySQL: 3306
- Redis: 6379

A.2 Authentication

- JWT-based authentication is used
- Header: Authorization: Bearer <access_token>
- Token refresh: POST /api/auth/refresh

A.3 Core APIs Overview

- **Authentication APIs**: Signup, Signin, Refresh
- **User APIs:** Onboarding, Tools, Preferences
- **Dataset APIs:** Upload, Clean, Undo, Resume, Download
- **Training APIs**: Train models, Track jobs, Compare results
- **Health APIs**: System status check

A.4 System Components

- **ML Service**: Handles preprocessing and model training
- **WebSocket Events:** Real-time job updates (progress, completed, failed)
- **Database:** MySQL (users, datasets, pipelines, jobs)
- **Queue System**: Redis + BullMQ for asynchronous processing

A.5 Important Notes

- Only CSV datasets supported
- JWT required for secure APIs
- Asynchronous job processing is used

➢ Appendix B: Database Schema

B.1 Overview

The DataForge platform uses a MySQL database to manage users, datasets, pipelines, and training jobs.

B.2 Key Tables

- users – Stores user information and authentication details
- datasets – Stores dataset metadata
- pipelines – Stores pipeline configurations
- pipeline_steps – Stores preprocessing steps
- training_jobs – Stores job execution details
- trained_models – Stores trained model information

B.3 Relationships

- One user can have multiple datasets
- One pipeline contains multiple preprocessing steps
- One pipeline can have multiple training jobs

B.4 Notes

- Dataset content is not stored permanently (only metadata stored)
- Database ensures consistency and efficient tracking of pipelines and jobs